Detecting LLM-Generated Text: A Comparative Study of Fine-Tuned, Statistical, and Interpretable Detection Methods

Deogracias Mufula
MFLDEO001@myuct.ac.za
University of Cape Town
Cape Town, South Africa

Murema Manganyi
mngkho012@myuct.ac.za
University of Cape Town
Cape Town, South Africa

[Umar Parker]
PRKUMA001@myuct.ac.za
University of Cape Town
Cape Town, South Africa

Abstract

Current approaches to detecting machine-generated text achieve strong performance, but suffer from several important limitations. Most methods rely on black-box models that provide little insight into how decisions are made, often fail to generalise across domains and model types, and are largely restricted to English. These issues limit their reliability and applicability in real-world, multilingual settings.

In this project, we investigate multiple approaches to LLM-generated text detection, with a particular focus on the trade-off between predictive performance and interpretability. We construct a multilingual dataset by combining human-written text from MzansiText with machine-generated text produced by large language models, covering a range of South African languages.

We evaluate three detection approaches: a fine-tuned neural model, a statistical feature-based method, and a rationale-based model that performs classification using selected subsets of tokens, providing intrinsic explanations for its predictions. Models are evaluated using F1-score and AUROC, while interpretability is assessed using faithfulness metrics and human evaluation.

The aim of this project is to determine whether interpretable models can provide meaningful and reliable explanations for LLM-generated text detection without significantly reducing performance, particularly in multilingual and low-resource settings.

1 Introduction

Large language models (LLMs) such as GPT-4 [18] and Llama [23] can generate text that is often difficult to distinguish from human writing. This creates a need to determine whether a given piece of text was written by a human or produced by a model, a task referred to as LLM-generated text detection.

Current detection approaches achieve strong performance using machine learning models trained for binary classification [15]. However, three limitations remain. First, many models provide little insight into how predictions are made, making their behaviour difficult to interpret [19]. Second, their performance can degrade when applied to new domains or outputs from different language models [26]. Third, most methods are developed and evaluated primarily on English data, with limited attention to multilingual and low-resource settings [25].

We investigate these limitations through a comparison of different detection approaches. In particular, we focus on interpretability, generalisation, and multilingual robustness. Here, interpretability refers to the ability to identify which parts of the input text contribute to a model's prediction.

We consider three types of models. The first is a fine-tuned neural classifier, which learns to distinguish between human-written and machine-generated text directly from data. The second is a statistical feature-based approach that relies on measurable properties of the text, including token likelihood and perplexity [16]. The third is a rationale-based model, which performs classification using a subset of selected tokens, allowing those tokens to serve as an explanation for the prediction [4]. We also include a simple feature-based baseline to provide a consistent point of comparison across all approaches.

To evaluate these approaches, we construct a multilingual dataset by combining human-written text from MzansiText [14] with machine-generated text produced by large language models. This enables evaluation across multiple South African languages.

All models are evaluated using F1-score and AUROC for detection performance, and using faithfulness measures and human evaluation to assess interpretability. The aim is to determine how detection methods compare across these dimensions and whether interpretable approaches can provide useful explanations without significantly reducing performance.

2 Background and Related Work

These three approaches are not independent; rather, they reflect different assumptions about what signals distinguish machine-generated text. This project studies them jointly to understand the trade-offs between accuracy, generalisation, and interpretability under the same experimental conditions.

Existing approaches to detecting LLM-generated text fall into three categories: fine-tuned supervised models, statistical and zero-shot methods, and interpretable methods. These approaches differ in their assumptions, performance characteristics, and ability to generalise across domains. This project studies these three paradigms under a shared experimental framework, enabling direct comparison between accuracy-driven and interpretability-driven methods.

2.1 Fine-Tuned Detection Methods

Supervised Encoder Models for Detection: BERT[7] is a Transformer model that uses a bidirectional masked language modelling pre-training objective. What made BERT stand out from previous unidirectional models is that BERT's bideirectional encoder looks at both the left and right context to predict the token. This gives the model better understanding of the context of the sentence. This makes it good for classifying text that requires the model to understand the context of a large text. BERT also demonstrated that it could easily be adapted to downstream tasks by adding a linear classification output layer.

mBERT[7] uses the same model architecture as BERT but is adapted to be used as a multilingual model. MBERT was trained on a corpus of 104 languages using Wikipedia data with a shared

WordPiece vocabular. While mBERT is outperformed by other more advanced models such as RoBERTa, it is widely used as a baseline in for cross-lingual benchmarks.

RoBERTa [13] is a model that improves upon the architecture of BERT by removing the Next Sentence prediction that BERT uses. The model was trained on a larger amount of data with bigger batches of data used. These changes improved the performance of the model on downstream tasks making RoBERTa a widely used model for text classification. The biggest downside is that RoBERTa based models is that they underperformed on data that is outside of the domain that they have been trained on [24].

XLM-RoBERTa [6]is a RoBERTa based model that is trained on 100 languages using CommonCrawl data. XLM-RoBERTa uses temperature based sampling to prevent English from dominating the training. This allows XLM-RoBERTa to learn shared cross-lingual representations across languages. While XLM-RoBERTa is able to detect text in different languages, it still struggles with data that is out of its training domain. This causes XLM-RoBERTa to struggle to detect texts in unseen languages. Since South African languages are not well represented in the data used to train XLM-RoBERTa, it represents these languages poorly.

Domain Adaptive pre-training and Language Adaptive pre-training: Domain Adaptive pre-training is a method to continue training of already pre-trained language models on a large corpus of unlabeled text from a specific domain [9]. DAPT continues the training using the model's objective. This allows a model to be adapted to more niche applications without creating a new model. Language Adaptive pre-training is when a pre-model continues training on a specific language. This aims to improve the models understanding of a language. These methods increase the performance of the model on downstream tasks especially when the data is far from the models previous training distribution. This is useful to us as South African languages are not well represented in training data.

LoRA:. LoRA [10]is a parameter-eficient fine-tuning strategy designed to adapt large pre-trained models to specific tasks without needing to train all the parameters in the model. It does this by freezing the pre-trained weights and injects rand decomposition matrices in each layer of the transformer architecture. The benefit of this is that it reduces the amount of parameters needing to be changed and reduces the amount of computing power needed to continue training the model. Models using LoRA perform as well as or better than models that are fully fine-tuned. LoRA acts as an implicit regularisor preventing overfitting.

Joint vs Per-Language fine-tuning: Joint fine-tuning is the process of fine-tuning a model on dataset of multiple languages creating one model that shares patterns across languagesMulcaire et al. [17]. Joint fine-tuning produces the best results when the languages are typologically related. The main disadvantage is that higher-resource languages tend to overpower lower-resource languages. This is relevant since many South African languages are low-resource and needs to be handled.

Per-Language fine-tuning creates a model for each language where each language is trained on a dataset only containing the specified language. This creates models that perform better on individual languages than joint fine-tuning[5]. Per-Language fine-tuning performs worse on low-resource languages as it does not share subword vocabulary. The MULTITuDE benchmark [15] found that multilingual detectors generalise better than monolingual detectors. Per-Language fine-tuning performs better when there are large amounts of data for the language which is not the case for all South African Languages.

2.2 Statistical and Zero-Shot Methods

Statistical and zero-shot detection methods approach the problem of LLM-generated text detection by analysing the probabilistic structure of text rather than relying on supervised training. These methods exploit the fact that machine-generated text is produced by a language model that assigns probabilities to tokens, which leads to measurable patterns in token likelihoods, ranks, and distributions. A central metric is perplexity, which reflects how predictable a sequence is under a given language model. Machine-generated text often exhibits lower perplexity under the generating model due to its optimisation for likelihood, while human text can display greater variability [8, 21].

Beyond perplexity, recent work has introduced more refined statistical signals, including per-token log-likelihood, token rank, entropy-based measures, and curvature-based methods such as DetectGPT, which evaluate, (this project focuses only on log-likelihood and curvature-based methods) how the likelihood of a text changes under small perturbations. These approaches do not require labelled training data and can therefore be applied in zero-shot settings, making them attractive for rapid deployment and for scenarios where annotated datasets are limited or unavailable. Additionally, they offer a degree of transparency, as their decisions are based on interpretable probabilistic signals rather than opaque learned representations [16, 27].

However, the literature highlights several limitations. Statistical detectors are sensitive to the choice of reference language model and may fail when the evaluation model differs significantly from the generator. They are also vulnerable to paraphrasing and rewriting attacks, which can alter token probabilities while preserving meaning, thereby weakening the detection signal. Furthermore, while these methods are more interpretable than neural classifiers, their outputs are still indirect, requiring careful thresholding and calibration to achieve reliable performance. As a result, statistical methods are best viewed as complementary to supervised approaches, offering robustness and transparency in some settings but lacking consistency under strong distribution shifts [16, 25, 26].

2.3 Interpretability in Detection Methods

Interpretability in text classification refers to the ability to understand which parts of an input influence a model's prediction. In LLM-generated text detection, this means identifying the linguistic or stylistic signals that lead to a classification decision.

Existing approaches to interpretability can be broadly grouped into post-hoc and ante-hoc methods. Post-hoc methods aim to explain predictions after they are made, typically using techniques such as feature attribution. While these methods can provide useful insights, they do not guarantee that the explanation faithfully reflects the model's internal reasoning [19]. Ante-hoc approaches,

in contrast, build interpretability directly into the model, allowing predictions to be understood from the model itself.

Feature-based approaches are a common example of ante-hoc interpretability. These methods rely on measurable properties of text, such as word usage, sentence structure, and n-gram patterns, and have been widely used in areas such as authorship attribution and stylometry [22]. In the context of LLM-generated text detection, similar features have been shown to capture differences in token distributions and stylistic patterns between human and generated text [2]. Other approaches, such as GLTR, provide interpretability by exposing token-level probability information in a human-readable form, offering insight into model behaviour without constraining the prediction process [8].

Rationale-based models extend this idea by selecting a subset of the input text and restricting predictions to that subset, allowing the selected tokens to serve as a direct explanation of the model's decision [4, 11]. These models offer a more tightly coupled form of interpretability, but have seen limited application in LLM-generated text detection.

Across these approaches, interpretability is often treated as secondary to detection performance. Existing benchmarks focus primarily on accuracy and generalisation, with limited attention given to whether predictions can be explained or verified. As a result, there is limited understanding of how different detection approaches compare in terms of their ability to provide transparent and trustworthy decisions.

3 Problem Statement and Research Questions

3.1 Problem Statement

The limitations observed across existing detection approaches, particularly their sensitivity to changes in language, generator model, and input text, have motivated this study. This project aims to compare fine-tuned neural models, statistical token-probability methods, and rationale-based interpretable models for detecting LLM-generated text by evaluating them under a shared multilingual experimental setup, examining how these approaches differ in detection performance, their ability to maintain accuracy when the language or generator model changes, and in certain cases, the ability to provide faithful explanations

3.2 Research Questions

3.2.1 Fine-tuned detection models.

How well does a fine-tuned XLM-RoBERTa model compare to a basic logistic regression model when detecting machine-generated text across South African Languages as measured by macro-F1 score.

3.2.2 Statistical detection methods.

How well do token-probability-based statistical methods, including per-token log-likelihood and curvature-based scoring, compare with a simple logistic regression binary classifier?

3.2.3 Interpretable detection methods.

How effectively can a rationale-based classifier detect LLM-generated text, and to what extent can it provide explanations that are faithful to the model and interpretable by human evaluators?

4 Methodology

4.1 Overall Experimental Design

We study LLM-generated text detection as a binary classification task using the MzansiText multilingual dataset. We consider three approaches: a fine-tuned multilingual neural classifier, statistical token-probability-based methods, and a rationale-based interpretable classifier.

We apply all approaches to the same task and dataset, the difference is in how inputs are processed and how predictions are produced. The fine-tuned model is trained on full-text inputs, the statistical methods operate without additional training using token-level scoring functions, and the interpretable model performs classification based on selected subsets of the input text.

Evaluation is conducted using a common set of metrics, detection performance is measured using F1 score and AUROC, and generalisation is assessed across multiple languages and generator models. For the interpretable model, we also evaluate the selected rationales to see if they are sufficiently interpretable by human evaluators.

4.2 Detection Approaches

4.2.1 Fine-Tuned Models

Model: The model that we will use is XLM-RoBERTa [6]. XLM-RoBERTa is a transformer encoder model that is based on RoBERTa [12] which is a derivative of BERT [7] that improves upon it by removing the next-sentence prediction objective. XLM-RoBERTa is a Masked language model which means that the model is trained to predict hidden tokens in a sequence. XLM-RoBERTa was trained on 2.5TB of filtered CommonCrawl data across 100 languages that allows it to learn shared cross-lingual representations. This model will be used with a linear classification head added to the encoder's `[CLS]` token for binary classification. As a baseline we will also be fine-tuning an mBERT model [7] under the same conditions as XLM-RoBERTa as mBERT is a widely used multilingual encoder while also allowing us to to test the benefit of a RoBERTa based training approach.

Pre-Training: Before we do any classification training, XLM-RoBERTa will undergo continuous masked language model pre-training on the South African languages following the domain-adaptive pre-training framework of Gururangan et al. [9] and the language-adaptive pre-training approach of Alabi et al. [1]who found that continued Masked Language Model pre-training increases the performance of downstream tasks for languages that are underrepresented in XLM-RoBERTa's original training data. We will use data from the MzansiText[14] corpus to train the model. We will use XLM-RoBERTa's sentencePiece tokenizer to convert the text into tokens. The languages will be jointly trained. To prevent higher resource languages from dominating, we will use temperature based sampling. This model will be used as the base for fine-tuning.

Joint vs Per-model fine-tuning: An important decision for multilingual Natural Language Processing is whether to train a single model jointly on all languages or to train seperate models for each language[6].

Joint fine-tuning is the process of training a model on a dataset of multiple languages simultaneously whereas Individual fine-tuning only uses a dataset containing a single dataset at a time. The reason we chose Joint fine-tuning is that it allows the model to learn shared patterns across languages Mulcaire et al. [17]. This benefits low-resource languages through cross-lingual transfer for related language families. This is cConsistent with the MULTITuDE results that have shown that multilingual detectors generalise better to unseen languages than monolingual detectors [15]. Some downsides of joint fine-tuning is that the model may bias towards high-resource languages which is why we use temperature based sampling across the languages as shown when training MzansiLM [14].

Per-model fine-tuning creatures separate models for each language based on the base model trained. Thise fine-tuning is only uses the labelled data for the language being trained. This produces specialised models that can outperform joint fine-tuned models Chung et al. [5]. This requires there to be sufficient data available otherwise overfitting can occur.

Fine-Tuning setup: We will use LoRA [10] to fine-tune the model for classification using the PEFT library. LoRA works by freezing all pre-trained weights and injecting trainable low rank decomposition matrices into each layer of the transformer architecture. LoRA is useful as it decreases the amount of parameters that needs to be trained relative to full fine-tuning and decreases the amount of GPU memory needed during training making it possible to train under limited computing resources. LoRA also acts as an implicit regulariser reducing overfitting on small language datasets. LoRA gives results similar or better to doing full fine-tuning making it worthwhile to implement to save time and resources.

4.2.2 *Statistical Methods: Log-Likelihood and Curvature-Based Detection*

The core intuition behind statistical detection is direct: when a language model generates text, it selects high-probability tokens at each step, leaving a measurable imprint in the sequence of token probabilities. Human writing, unconstrained by any model's probability distribution, exhibits more variability in those same probability signals. This project exploits that difference without requiring any labelled training data.

We implement two complementary zero-shot detectors that operate on the same underlying signal but capture different aspects of it: (i) a score-based detector that measures how predictable a text is under the scoring model, using average per-token log-likelihood as its score, and (ii) a curvature-based detector that measures whether a text sits near a local maximum of the model's probability surface, using perturbation discrepancy as its score. Together, these two detectors test whether the generation process is detectable from global probability mass alone, and whether sensitivity to small rewrites provides a complementary and more robust signal.

All detectors operate on token probabilities extracted from a single open-weight scoring model, GPT-2 XL (1.5B parameters). This choice is necessary because closed-source models rarely expose token-level logits, and using a fixed backbone ensures that performance differences arise from the detection method rather than from the scoring model [8, 16, 21].

Statistical Signal Extraction. Both detectors derive their scores from the same token-level probability sequence produced by GPT-2 XL. We define the two core signals below, stating the intuition before the formal expression in each case.

- **Log-likelihood.** Machine-generated text tends to occupy high-probability regions of the scoring model's distribution because the model is trained to maximise the likelihood of its own outputs. Human writing is not constrained in this way, so it tends to be less predictable under the model. The length-normalised average log-likelihood captures this difference as a scalar score:

$$\ell(x) = \frac{1}{n}\sum_{i=1}^{n} \log p_\theta(t_i \mid t_{<i}),$$

  where higher values signal that the text is more predictable under the model [21]. Perplexity, defined as $\mathrm{PPL}(x) = \exp(-\ell(x))$, gives the reciprocal view of the same signal and is reported as a secondary diagnostic to facilitate comparison with earlier work [8].

- **Curvature (perturbation discrepancy).** The log-likelihood detector captures global probability mass but misses a structural property that curvature-based methods exploit: machine-generated text tends to sit near local maxima of the model's log-probability surface. This means that small, meaning-preserving rewrites cause its log-probability to drop more sharply than they do for human text. Following DetectGPT [16], we formalise this drop as:

$$d(x, M_\theta, q) = \log p_\theta(x) - \mathbb{E}_{\tilde{x}\sim q(\cdot|x)} \log p_\theta(\tilde{x}),$$

  where the expectation is taken over meaning-preserving perturbations of x. A large positive value of d indicates that the original text is more probable than its rewrites, which is a hallmark of machine-generated text.

Average Per-Token Log-Likelihood Detector. This detector is the simpler of the two and provides the most direct measure of how well the scoring model predicts the observed text.

Detection score.
The length-normalised average log-likelihood $\ell(t)$ is used directly as the detection score:

$$\ell(t) = \frac{1}{n}\sum_{i=1}^{n} \log p_\theta(t_i \mid t_{<i}).$$

Higher values indicate that the text sits in a high-probability region of the model's distribution, characteristic of machine-generated text [8, 21].

Auxiliary signal.
The variance of token log-likelihoods is retained as an auxiliary indicator. Human text often exhibits larger variability in per-token probability, so comparing the variance alongside the mean provides a richer characterisation of each sample. The variance is used in exploratory analysis but does not contribute to the scalar detection score.

Decision rule.
A threshold is selected on the validation set by maximising

either Youden's J statistic (J = sensitivity + specificity − 1) or the validation F1 score. The threshold is then frozen before any test-set evaluation.

Curvature-Based Detector (DetectGPT-Style). The curvature-based detector captures the structural property that score-based methods miss: whether the text sits near a local maximum of the log-probability landscape. This follows the perturbation-discrepancy paradigm of DetectGPT [16] and Fast-DetectGPT [3].

Perturbation protocol.
Given a passage x, we generate $k = 100$ meaning-preserving perturbations $\tilde{x}_1, \ldots, \tilde{x}_k$ using T5-small as a mask-filling model. Random two-word spans are masked until 15% of the passage's tokens are replaced, and T5-small fills each span by sampling. The raw perturbation discrepancy is:

$$d(x, M_\theta, q) = \log p_\theta(x) - \mathbb{E}_{\tilde{x}\sim q(\cdot|x)} \log p_\theta(\tilde{x}),$$

approximated by the sample mean $\tilde{\mu} = \frac{1}{k}\sum_i \log p_\theta(\tilde{x}_i)$. We normalise by the perturbation standard deviation to improve discriminability across texts of varying length:

$$\hat{d}_x = \frac{\log p_\theta(x) - \tilde{\mu}}{\tilde{\sigma}_x}, \tag{1}$$

where $\tilde{\sigma}_x$ is the standard deviation across the k perturbation log-probabilities.

Intuition.
Machine-generated text sits near local maxima of the log-probability surface. Meaning-preserving rewrites move the text away from these maxima, causing a sharper drop in log-probability than typically observed for human text. The normalised discrepancy $\hat{d}_x$ captures this local sensitivity [3, 16].

Decision rule.
A threshold is selected on the validation set using Youden's J or maximum validation F1, then applied unchanged to the test set.

Calibration and Threshold Selection. Both detectors follow a strict *validation-only* calibration protocol to prevent test leakage:

- For each detector, a scalar threshold is selected on the validation split by maximising either Youden's J statistic or the validation F1 score. The selection criterion is recorded and held consistent across experiments.
- The test set is held out entirely from threshold tuning. Once the threshold is fixed, test-set metrics (accuracy, precision, recall, F1) are computed and reported.
- AUROC serves as the primary comparative metric because it is threshold-independent. F1 and accuracy complement it at the calibrated operating point.

Statistical Validity and Uncertainty Analysis. Detector comparisons are based on paired evaluation over the same test items, ensuring that observed differences reflect genuine methodological contrasts rather than sampling variation. For each pair of detectors we report:

- **Bootstrap confidence intervals** (1000 replicates) for AUROC, F1, accuracy, precision, and recall.
- **McNemar's test** to assess whether classification disagreements between the two detectors on the same samples are statistically significant.
- **Paired bootstrap comparison of AUROC differences**, which resamples test instances jointly and tests whether the observed gap exceeds chance variation.

Robustness Testing. We evaluate each detector under three types of distribution shift, treating robustness tests as targeted stress experiments rather than pass/fail summaries:

(1) **Domain shift** — evaluating when the text genre (news, government documents, web content) changes between calibration and test.
(2) **Generator shift** — testing on text produced by an LLM not used during threshold selection (e.g., Google Gemini).
(3) **Surface-form perturbations** — applying paraphrasing, word-substitution attacks, and text-length reduction before rescoring.

For each stress test, we report the absolute drop in AUROC and F1, the shift in the optimal calibration threshold, and the change in score separation between the human and machine-text distributions. A key hypothesis is that the curvature detector degrades more slowly than the log-likelihood detector under paraphrasing, since curvature captures local structure rather than absolute probability mass [16].

Error Analysis. To complement aggregate metrics, we perform a manual error analysis at the optimal threshold for each detector, inspecting:

- **False positives:** texts that are short, highly structured (e.g., government forms), formal, or written by non-native speakers. These are prone to misclassification because they exhibit statistical regularities similar to machine-generated text.
- **False negatives:** paraphrased or low-probability machine-generated passages that have been deliberately altered to suppress the generation signature.

This analysis makes concrete the practical risks of deploying statistical detectors and informs recommendations for their safe use.

Distribution Visualisation. A key strength of the statistical branch is that its decisions can be traced back directly to token-probability signals. We will produce:

- **Histograms and kernel density estimates (KDEs)** of log-likelihood and curvature scores, overlaid for the human and machine classes. These plots expose the degree of class overlap and guide threshold selection.
- **Token-level probability heatmaps** for selected samples, highlighting which tokens the scoring model treats as highly probable and how that pattern differs between human and machine text.
- **Perturbation-discrepancy visualisations** that illustrate the log-probability drop after meaning-preserving rewrites, connecting the curvature detector's output to observable textual changes.

These visualisations make the statistical branch a transparent, interpretable counterpart to the fine-tuned and rationale-based components.

Expected Limitations. The log-likelihood detector is sensitive to the choice of scoring model and can be weakened by paraphrasing attacks that raise the surface probability of human text. The curvature detector is more robust to paraphrasing but computationally more expensive, as it requires generating $k = 100$ perturbations per sample. Both detectors may exhibit language-specific biases due to tokenisation differences across South African languages. Results will be reported with explicit caveats about these conditions, framing limitations as empirical findings rather than weaknesses of the approach.

Relation to Earlier Work. This branch builds directly on the progression from raw log-likelihood signals [8, 21] to DetectGPT [16] and Fast-DetectGPT [3]. By implementing both detectors under a shared scoring backbone and rigorous evaluation protocol, we contribute a focused empirical comparison of whether global probability mass or local probability curvature better separates human from machine-generated text across South African languages and unseen generator models.

4.2.3 *Interpretable Detection via Rationale-Based Models*

We use a rationale-based classification model to perform LLM-generated text detection while providing intrinsic explanations. Instead of using the full input, the model selects a subset of tokens and makes its prediction using only this subset. The selected tokens serve as the explanation for the model's decision.

Let $x = (x_1, x_2, \ldots, x_n)$ denote an input sequence. The model consists of two components: a *selector* and a *predictor*. The selector assigns a relevance score to each token and defines a distribution over binary masks $z = (z_1, \ldots, z_n)$, where $z_i \in \{0, 1\}$ indicates whether token x_i is retained. The selected subset is given by $x \odot z$.

The predictor operates on the masked input and produces a binary classification:

$$\hat{y} = f(x \odot z).$$

Intuitively, $x \odot z$ corresponds to keeping only the selected tokens and discarding the rest of the input. The predictor therefore acts as a standard classifier, but is forced to make its decision using only this reduced subset.

In practice, the selector is implemented as a lightweight neural module that produces a selection probability for each token. A mask is then sampled from these probabilities, and only the selected tokens are passed to the predictor. The predictor is a neural classifier that learns to distinguish between human-written and machine-generated text based on the masked input.

The model is trained end-to-end using labelled examples. The predictor is trained to make correct classifications, while the selector is implicitly guided by this objective. If the selector removes tokens that are important for the decision, the predictor's performance drops. Over time, this pushes the selector to retain tokens that are useful for distinguishing between the two classes.

This gives the model intrinsic interpretability. Since the prediction is based only on the selected tokens, these tokens directly reflect the information used to make the decision, rather than being added as a separate explanation step.

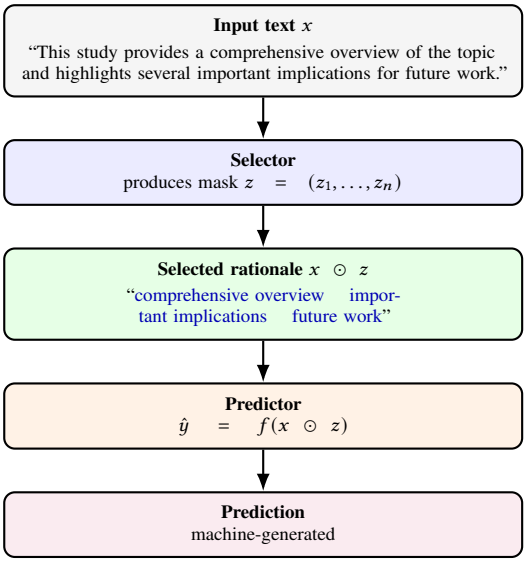


Figure 1: Rationale-based detection. The selector produces a binary mask over the input text, giving the rationale $x \odot z$. The predictor then classifies the text using only this selected subset, so the same tokens serve as both input and explanation.

Figure 1 illustrates this process with a concrete example, showing how the selected tokens form both the input to the predictor and the explanation for the final decision.

In this project, we apply this framework to the task of LLM-generated text detection in a multilingual setting. The selected tokens are analysed to understand what signals the model relies on when distinguishing between human-written and machine-generated text, and whether these signals are consistent across languages.

This connects directly to our evaluation. Because the model relies only on the selected tokens, we can measure how important they are using sufficiency and comprehensiveness. We also include a human evaluation to assess whether the selected tokens form explanations that are clear and meaningful to readers.

4.3 Evaluation

4.3.1 *Classification Performance*

We will evaluate all three approaches on the task of classifying input text as either human-written or machine-generated. Performance is assessed using precision, recall, F1-score, and AUROC.

Precision in this setting is the proportion of texts predicted as machine-generated that are correctly identified, while recall is the proportion of machine-generated texts that are correctly detected by the model. These metrics can be expressed in terms of true positives (TP), false positives (FP), and false negatives (FN) as follows:

$$\text{Precision} = \frac{TP}{TP + FP} \tag{2}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{3}$$

The F1-score combines precision and recall into a single metric using the harmonic mean:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4)$$

Precision captures how often texts flagged as MGT are truly machine-generated, while recall captures how many MGTs are successfully detected. The F1-score balances the two, penalising models that achieve high precision at the cost of recall, or vice versa.

AUROC (Area Under the Receiver Operating Characteristic curve) measures how well the model ranks MGT higher than human-written text across all decision thresholds. It corresponds to the probability that a randomly selected machine-generated text is assigned a higher score than a randomly selected human-written text. A higher AUROC therefore indicates better separation between the two classes [16].

4.3.2 Multilingual Performance and Generalisation

We compute all metrics on the held-out test split of the MzansiText dataset. Results are reported per language and averaged across languages to reflect the multilingual setting.

To assess generalisation, we evaluate performance on text generated by models not seen during training and compare this to performance on generators observed during training. This allows us to measure how well each approach transfers across different generation sources and linguistic settings.

4.3.3 Interpretability and Explanation Evaluation

For the interpretable model, we evaluate the extracted rationales using sufficiency and comprehensiveness [4].

Sufficiency is measured by retaining only the selected tokens and computing the change in the model's prediction confidence. A rationale is considered sufficient if the model produces a similar prediction using only the selected tokens.

Comprehensiveness is measured by removing the selected tokens from the input and observing the resulting drop in prediction confidence. Larger drops indicate that the removed tokens were important for the prediction.

In addition, we conduct a human evaluation study. Participants are presented with input texts, model predictions, and corresponding highlighted tokens. These tokens represent the model's explanation for its decision.

Participants are asked to rate, on a Likert scale from 1 (not informative) to 5 (highly informative), how well the highlighted text explains the model's prediction. The final score is computed as the average rating across participants and examples, providing a human-centred measure of explanation quality.

4.3.4 Shared Baseline

To provide a consistent and interpretable point of comparison across all approaches, we include a simple feature-based baseline model. The baseline represents each input text using n-gram features and performs binary classification using logistic regression.

In this context, an n-gram refers to a sequence of consecutive words. For example, a unigram corresponds to a single word, while a bigram captures pairs of adjacent words. These features allow the model to capture simple patterns in word usage and phrasing that may differ between human-written and machine-generated text.

Each text is converted into a representation based on the frequency of these word sequences. The logistic regression classifier then uses these features to predict whether the text is human-written or machine-generated.

The model is trained and evaluated using the same data splits and settings as the other approaches, ensuring that differences in performance reflect differences in modelling rather than experimental setup.

As a simple approach that is not tied to any particular model type, this baseline does not rely on complex architectures or training strategies. This makes it suitable as a common reference point across all detection methods considered in this work, allowing for a consistent comparison of performance and behaviour across fundamentally different modelling approaches.

4.4 Dataset

We will construct a binary-labelled corpus for this project, as no multilingual MGT detection dataset covers all eleven official South African languages. The corpus will combine human-written text drawn from MzansiText [14] with machine-generated text produced by prompting large language models, yielding samples labelled $y = 0$ for human-written and $y = 1$ for machine-generated.

MzansiText is a curated multilingual corpus compiled from public web sources and processed through a deterministic quality-filtering and deduplication pipeline [14]. We will sample text from it across languages and, where possible, across domains. For each sampled document, we will produce a corresponding machine-generated text by prompting one or more LLMs using prompts derived from the same domain and topic context. This ensures that human and generated samples remain comparable, so that the classification signal reflects authorship rather than subject matter.

A central motivation for constructing the corpus is language coverage. Most existing detection datasets are English-only or cover a small number of high-resource languages. MzansiText includes lower-resource languages such as Tshivenda, Xitsonga, and Siswati alongside higher-resource ones such as Afrikaans and isiZulu, and the constructed corpus will preserve this breadth. This makes it possible to evaluate detector performance across the full range of languages rather than treating multilingual evaluation as an afterthought.

5 Ethical, Professional, and Legal Issues

This project includes a human evaluation component in which participants will assess the quality of model-generated rationales. We will therefore apply for ethics clearance from the Inter-Faculty Human Research Ethics Committee at the University of Cape Town before conducting this study. Participation will be voluntary and based on informed consent, and all responses will be anonymised.

The remaining experimental work will use publicly available corpora and text generated under controlled conditions. All datasets, models, and third-party resources will be properly acknowledged, and their respective licensing conditions will be followed. No personally identifiable information will be collected or used in this study.

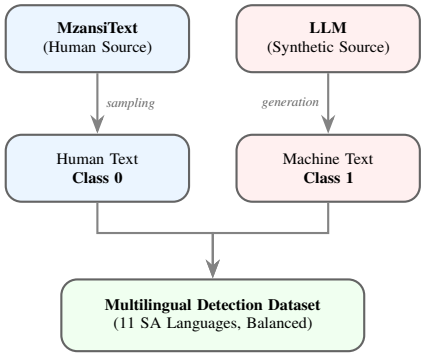


Figure 2: Dataset construction procedure. Human-written text is sampled from MzansiText and combined with domain-matched LLM-generated text to form a binary-labelled corpus.

5.1 Anticipated Outcomes

Detection Approaches. We expect fine-tuned neural detectors to achieve the highest classification accuracy on in-domain data, but to exhibit reduced performance under distribution shift, particularly on unseen generators, domains, or paraphrased text.

We expect statistical methods to demonstrate more stable cross-domain performance due to their reliance on model-agnostic features, although with lower overall accuracy.

We expect interpretable approaches to produce transparent and verifiable decisions. In particular, rationale-based models are expected to identify concise subsets of text that reflect stylistic differences between human and machine-generated content, although this constraint may limit classification accuracy.

Success Criteria. The project will be considered successful if all approaches are implemented within a shared experimental framework, meaningful comparisons can be made across accuracy and generalisation, and interpretable models produce rationales that achieve strong sufficiency and comprehensiveness scores.

Impact. This work will provide a comparative understanding of detection approaches under realistic conditions and establish a baseline for future research on interpretable LLM-generated text detection. It will also highlight the role of interpretability in supporting more transparent and trustworthy detection systems.

6 Project Plan

6.1 Work Allocation

Each team member is responsible for one detection approach, while core components of the project are developed collaboratively.

Shared Responsibilities

- Dataset construction and preprocessing
- Implementation of the shared baseline model
- Evaluation design and implementation
- Analysis of results
- Report writing and final deliverables

Fine-Tuned Models (Umar Parker)

- Implementation and training of the neural classifier
- Cross-language and cross-generator evaluation

Statistical Methods (Deogracias Mufula)

- Implementation of probability-based detectors
- Feature extraction and analysis

Interpretable Methods (Murema Manganyi)

- Implementation of rationale-based model
- Evaluation of explanation quality

6.2 Timeline and Milestones

The project follows a structured timeline from proposal submission to final presentation, as summarised in Table 1 and illustrated in the Gantt chart (Appendix A).

Table 1: Project milestones.

Date	Milestone
5 May 2026	Project proposal due
6 May 2026	Dataset preparation begins
19 May 2026	Model implementation begins
28 July 2026	Progress demonstration
10 August 2026	Paper scaffold due
24 August 2026	Full draft due
31 August 2026	Final paper submission
14 September 2026	Project report due
22 September 2026	Final demonstration
28 September 2026	Poster due
5 October 2026	Project website due

6.3 Deliverables

The primary deliverable is the academic paper. Additional deliverables are:

- Project proposal (this document)
- Progress demonstration with preliminary results from all three components
- Draft academic paper submitted for supervisor feedback
- Source code for all three detection components with documentation
- Final academic paper
- Final project demonstration
- Project poster
- Project website

6.4 Resources Required

Compute. We can run development and small-scale experiments on personal laptops. However full model training, particularly for the fine-tuned RoBERTa classifier and the rationale extraction model, requires GPU resources accessible via the UCT HPC cluster.

Software. All components use Python with PyTorch and the HuggingFace Transformers library. The interpretable component adapts the open-source codebase of Bastings et al. [4]. Interpretability metrics are computed using the `captum` library.

Data. The project uses a custom multilingual dataset constructed from MzansiText [14] and machine-generated text produced by large language models. Additional benchmark datasets may be used for supplementary comparison where appropriate. LLM API access is

required to generate South African language samples, as the project scales, the costs of the API calls goes up as well. A budget should be set aside for this, along with a strict usage policy to minimise costs.

Ethics. Standard publicly available benchmarks do not require ethics review. If any data collection goes beyond existing open-licence releases, the group will apply for clearance from the Inter-Faculty Human Research Ethics Committee.

7 Conclusion

This proposal addresses the growing difficulty of distinguishing AI-generated text from human writing by comparing three complementary approaches under a shared evaluation framework. The statistical branch examines whether token-level log-likelihood, perplexity, token rank, curvature, and token probability sequences contain enough signal to detect machine-generated text reliably across South African languages. The neural branch evaluates how well a fine-tuned XLM-RoBERTa model generalises across languages and generators, while the interpretable branch investigates whether faithful rationales can be produced without sacrificing too much predictive performance. Together, these components aim to produce a rigorous comparison of accuracy, generalisation, robustness, and interpretability, and to identify where each method works best and where it fails [8, 15, 16, 21, 25–27].

References

[1] Jesujoba O Alabi, David Ifeoluwa Adelani, Marius Mosbach, and Dietrich Klakow. 2022. Adapting Pre-trained Language Models to African Languages via Multilingual Adaptive Fine-Tuning. In *Proceedings of COLING*. https://aclanthology.org/2022.coling-1.382/

[2] Christopher M.J. André, Helene F.L. Eriksen, Emil J. Jakobsen, Luca C.B. Mingolla, and Nicolai B. Thomsen. 2023. Detecting AI Authorship: Analyzing Descriptive Features for AI Detection. In *Proceedings of the Seventh Workshop on Natural Language for Artificial Intelligence (NL4AI 2023)*.

[3] Guangsheng Bao, Yuxuan Zhao, Zhiyang Teng, Lin Yang, and Yue Zhang. 2024. Fast-DetectGPT: Efficient Zero-Shot Detection of Machine-Generated Text via Conditional Probability Curvature. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.

[4] Jasmijn Bastings, Wilker Aziz, and Ivan Titov. 2019. Interpretable Neural Predictions with Differentiable Binary Variables. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. 2963–2977.

[5] Hyung Won Chung, Thibault Fevry, Henry Tsai, Melvin Johnson, and Sebastian Ruder. 2021. Rethinking Embedding Coupling in Pre-trained Language Models. In *Proceedings of ICLR*. https://openreview.net/forum?id=xpFFI_NtgpW

[6] Alexis Conneau. [n. d.]. Unsupervised cross-lingual representation learning at scale. doi:10.18653/v1/2020.acl-main.747 Retrieved April 22, 2026 from.

[7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*. 4171–4186.

[8] Sebastian Gehrmann, Hendrik Strobelt, and Alexander M. Rush. 2019. GLTR: Statistical Detection and Visualization of Generated Text. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*. 111–116.

[9] Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A Smith. 2020. Don't Stop Pretraining: Adapt Language Models to Domains and Tasks. In *Proceedings of ACL*. https://aclanthology.org/2020.acl-main.740/

[10] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *Proceedings of ICLR*. https://openreview.net/forum?id=nZeVKeeFYf9

[11] Tao Lei, Regina Barzilay, and Tommi Jaakkola. 2016. Rationalizing Neural Predictions. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*. 107–117.

[12] Yinhan Liu. 2019. Roberta: A robustly optimized Bert pretraining approach. doi:10.48550/arXiv.1907.11692 Retrieved March 18, 2026 from.

[13] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv preprint arXiv:1907.11692* (2019).

[14] Anri Lombard. 2026. Mzansitext and MZANSILM: An open corpus and decoder-only language model for South African languages. https://arxiv.org/abs/2603.20732 Retrieved April 22, 2026 from.

[15] Dominik Macko, Robert Moro, Adaku Uchendu, Jason Lucas, Michiharu Yamashita, Matus Pikuliak, Ivan Srba, Thai Le, Dongwon Lee, Jakub Simko, and Maria Bielikova. 2023. MULTITuDE: Large-Scale Multilingual Machine-Generated Text Detection Benchmark. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 9960–9987.

[16] Eric Mitchell, Yoonho Lee, Alexander Khazatsky, Christopher D. Manning, and Chelsea Finn. 2023. DetectGPT: Zero-Shot Machine-Generated Text Detection using Probability Curvature. In *Proceedings of the 40th International Conference on Machine Learning*. 24950–24962.

[17] Phoebe Mulcaire, Zdeněk Kasner, and Noah A Smith. 2019. Polyglot Contextual Representations Improve Crosslingual Transfer. In *Proceedings of NAACL*. https://aclanthology.org/N19-1116/

[18] OpenAI. 2024. GPT-4 technical report. doi:10.48550/arXiv.2303.08774 Retrieved April 22, 2026 from.

[19] Cynthia Rudin. 2019. Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead. *Nature Machine Intelligence* 1, 5 (2019), 206–215.

[20] Areg Mikael Sarvazyan, José Ángel González, Paolo Rosso, and Marc Franco-Salvador. 2023. Supervised Machine-Generated Text Detectors: Family and Scale Matters. In *International Conference of the Cross-Language Evaluation Forum for European Languages*. 121–132.

[21] Irene Solaiman, Miles Brundage, Jack Clark, Amanda Askell, Ariel Herbert-Voss, Jeff Wu, Alec Radford, Gretchen Krueger, Jong Wook Kim, and Sarah Kreps. 2019. Release Strategies and the Social Impacts of Language Models. https://arxiv.org/abs/1908.09203 arXiv preprint arXiv:1908.09203.

[22] Efstathios Stamatatos. 2009. A Survey of Modern Authorship Attribution Methods. *Journal of the American Society for Information Science and Technology* 60, 3 (2009), 538–556.

[23] Hugo Touvron. 2023. Llama: Open and efficient foundation language models. doi:10.48550/arXiv.2302.13971 Retrieved April 22, 2026 from.

[24] Yuxia Wang. 2024. M4: Multi-generator, multi-domain, and multi-lingual black-box machine-generated text detection. https://arxiv.org/abs/2305.14902 Retrieved March 23, 2026 from.

[25] Junchao Wu, Shu Yang, Runzhe Zhan, Yulin Yuan, Lidia Sam Chao, and Derek Fai Wong. 2025. A Survey on LLM-Generated Text Detection: Necessity, Methods, and Future Directions. *Computational Linguistics* 51, 1 (2025), 275–338.

[26] Junchao Wu, Runzhe Zhan, Derek F. Wong, Shu Yang, Xinyi Yang, Yulin Yuan, and Lidia S. Chao. 2024. DetectRL: Benchmarking LLM-Generated Text Detection in Real-World Scenarios. In *Advances in Neural Information Processing Systems*, Vol. 37. 100369–100401.

[27] Yilun Xu, Yue Wang, Yaqing Wang, Yifan Bi, Huan Cao, Zhen Lin, Feifei Zhao, and Fei Wu. 2025. Training-free LLM-generated Text Detection by Mining Token Probability Sequences. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

A Risk Management

Table 2 presents the key risks facing the project along with their assessed probability and impact and the group's mitigation strategy for each.

Table 2: Risk assessment. P = probability, I = impact: H = high, M = medium, L = low.

Risk	P	I	Mitigation
Dataset not confirmed before preprocessing deadline (06/05)	M	H	Three ranked fallback datasets are identified. Preprocessing begins on whichever is confirmed first; the pipeline is dataset-agnostic.
Training instability in the HardKuma selector (gradient variance, non-convergence)	M	M	Stage 1, which uses a simpler L_0 Lagrangian objective, is a self-contained fallback result. The open-source reference codebase of Bastings et al. [4] is available as a starting point.
IB objective does not improve over Stage 1	L	L	Stage 1 is already a valid standalone contribution. The IB comparison is treated as an extension and reported as future work if unsuccessful.
Cross-domain performance degradation across all three models	H	L	This is an expected finding based on prior benchmarks [20, 26] and is framed as a research result. The shared evaluation setup makes the comparison informative regardless of absolute performance.
HPC cluster access delayed or unavailable	L	M	Early-stage training runs on personal machines. GPU-intensive jobs are scheduled from June onwards, giving sufficient lead time to obtain access. If access is not granted, training is distributed across group members' machines.
Group member unable to continue	L	H	Each component is fully independent of the others. The remaining members continue their own streams and the final paper acknowledges the incomplete component. The shared dataset and evaluation framework remain usable for the components that are completed.
Chosen dataset is too domain-specific for cross-domain evaluation	M	M	MULTITuDE explicitly covers multiple domains and languages [15]. If the confirmed dataset is single-domain, we construct a cross-domain split from multiple publicly available sources.

B Project Timeline

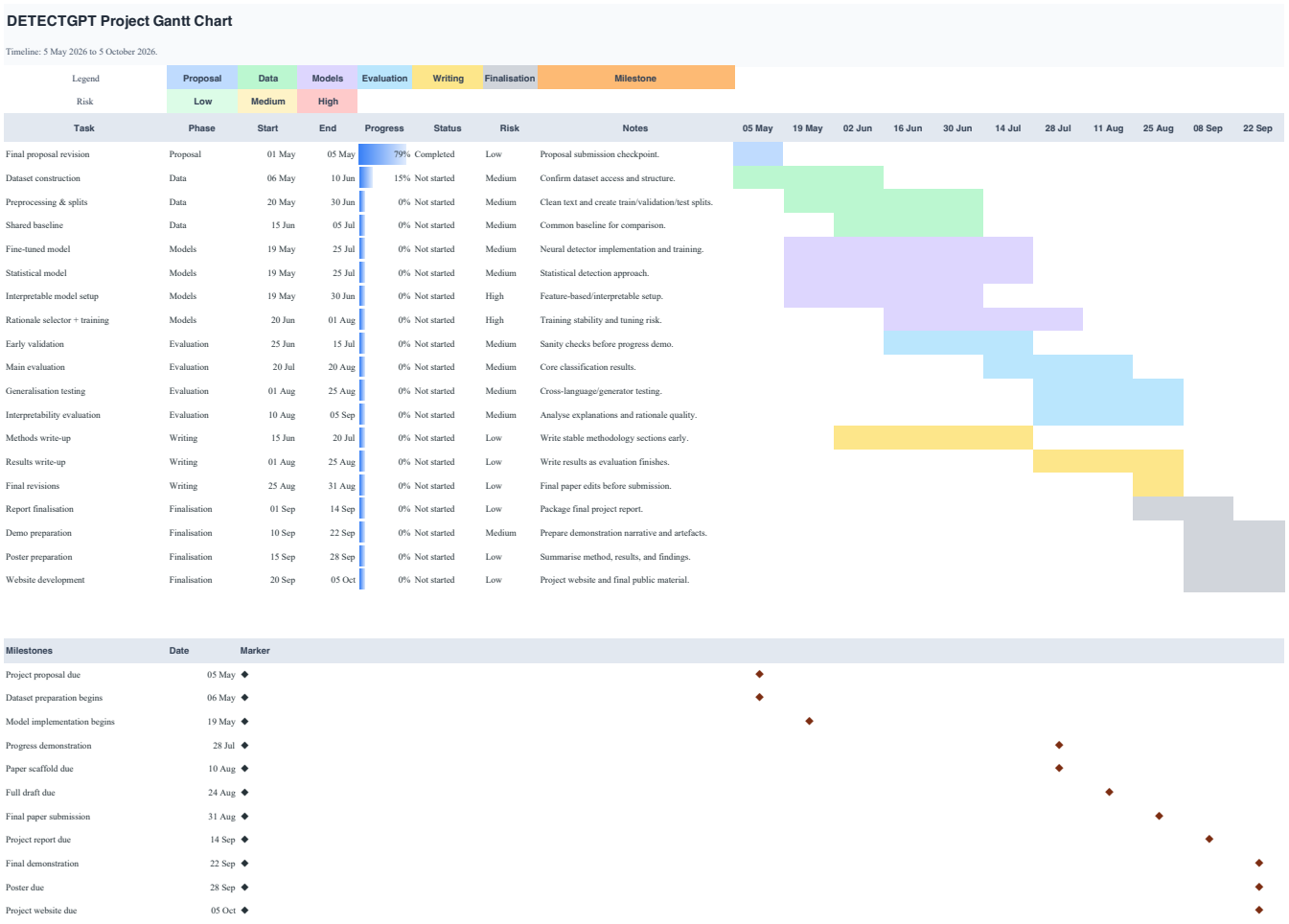


Figure 3: Project timeline illustrating the sequencing of phases, including data preparation, model implementation, evaluation, and final deliverables.